



CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

The Harness, Not The Chatbot

Claude Code deep dive, episode 1

Same prompt. Same project. Brilliant on Monday, confidently wrong on Friday.

Most people install Claude Code and treat it as a chat window that happens to sit in a terminal. That model of what is happening puts a ceiling on what you get out of it.

The documentation is precise: Claude Code is the agentic harness around Claude. The model reasons, the harness acts. You are not talking to a model, you are operating a harness, and every technique in this course follows from that one shift.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- What Claude Code actually is, in the documentation's own words
- The agentic loop, and that you are inside it
- What "fix the failing tests" really does
- Installing it, on every platform
- Five surfaces, one engine
- Pricing and usage limits, September 2026
- Why the same prompt degrades during the day

Out of scope

- Skills, MCP and subagents, which are episodes 6, 7 and 8
- Enterprise gateway, policy and audit, which is episode 15
- Benchmarks and comparisons against other tools
- Anything about how much faster anyone works: no such measurement is claimed

This sheet is the written form of episode 1. Everything on it was checked against primary documentation before recording.

01 The numbers, as of September 2026

Free	\$0, Claude Code not included
Pro	\$20/month, or \$17 billed annually
Max	from \$100/month
Max usage	5x or 20x Pro per 5-hour session
Limit window	rolling 5 hours, plus weekly caps on paid plans
Shared pool	terminal and chat draw from ONE limit

These are interview assumptions and published figures read on the date above. They are not published figures from any provider beyond what is cited, and no claim is made about any particular deployment.



03 It is a harness, not a chat window

The distinction is not pedantry. A chat window answers; a harness acts, and the things that make it act well are not the words you type.

The model reasons. The harness provides the tools, the context management and the execution environment that turn a language model into a coding agent.

So the thing to optimise is not your phrasing. It is the conditions the answer gets produced under.

Claude Code serves as the agentic harness around Claude: it provides the tools, context management, and execution environment that turn a language model into a capable coding agent.

Claude Code docs, How Claude Code works, checked September 2026

Without tools a model can only respond with text; with tools it can read code, edit files, run commands and interact with external services.

Claude Code docs, How Claude Code works, checked September 2026

04 The agentic loop, and your place in it

Three phases that blend into each other: gather context, take action, verify results. Then repeat, until the task is done or you stop it.

A question might need only the first phase. A bug fix cycles through all three repeatedly.

And you are in the loop too. You can interrupt at any point to steer, add a fact, or send it in a different direction. Every senior habit in this course is a way of intervening at the right moment.

When you give Claude a task it works through three phases: gather context, take action, and verify results, and these phases blend together.

Claude Code docs, How Claude Code works, checked September 2026

You can interrupt at any point to steer Claude in a different direction, provide additional context, or ask it to try a different approach.

Claude Code docs, How Claude Code works, checked September 2026



05 What one line actually does

You type: fix the failing tests. Six things happen.

It runs the suite to see what is failing. Reads the error output. Searches for the relevant source. Reads those files. Makes the edit. Runs the tests again to verify.

Every result changed what it did next. That is the loop in action, and it is why this is not autocomplete.

The built-in tools fall into five categories: file operations, search, execution, web, and code intelligence.

Claude Code docs, How Claude Code works, checked September 2026

Each tool use returns information that feeds back into the loop, informing the next decision.

Claude Code docs, How Claude Code works, checked September 2026

06 Installing it, September 2026

macOS, Linux and WSL: curl the install script and pipe it to bash. Windows has a PowerShell one-liner. There is a Homebrew cask and a WinGet package, and apt, dnf and apk are supported.

ONE DETAIL WORTH KNOWING BEFORE YOU CHOOSE. Native installations update themselves in the background. Homebrew and WinGet do not, so upgrading is your job on those.

Homebrew also ships two casks: a stable one that runs about a week behind and skips releases with major regressions, and a latest one that gets versions as they ship.

Native installations automatically update in the background to keep you on the latest version.

Claude Code docs, Overview, checked September 2026

Homebrew installations do not auto-update; run brew upgrade to get the latest features and security fixes.

Claude Code docs, Overview, checked September 2026

The claude-code cask tracks the stable release channel, typically about a week behind, and claude-code@latest tracks the latest channel.

Claude Code docs, Overview, checked September 2026

07 Five surfaces, one engine

Terminal, VS Code and JetBrains, a desktop app, and the browser at claude.ai/code, plus the mobile apps.

The sentence that matters: each surface connects to the same underlying engine, so your CLAUDE.md, your settings and your MCP servers work across all of them.

You are not configuring four tools. You are configuring one and choosing where to sit.

Each surface connects to the same underlying Claude Code engine, so your repo's CLAUDE.md files, settings, and MCP servers work across all of them.

Claude Code docs, Overview, checked September 2026

08 Pricing and the pool that catches people out

Claude Code is included in every paid plan. The figures are in the table above and they are the most perishable thing on this sheet.

The part people miss is not the price. Claude Code and your ordinary chats draw from ONE pool: all activity in both counts against the same usage limits.

So the afternoon you spend asking questions in the chat app is the afternoon your terminal runs out. Limits reset on a rolling five hour window, and paid plans add a weekly limit on top.

Claude Code is included in all paid plans.

claude.com pricing and Claude Help Center, checked September 2026

Both Pro and Max plans offer usage limits that are shared across Claude and Claude Code, meaning all activity in both tools counts against the same usage limits.

claude.com pricing and Claude Help Center, checked September 2026

Usage limits reset on a rolling five-hour session window, and paid plans add weekly limits on top.

claude.com pricing and Claude Help Center, checked September 2026



09 Why the same prompt degrades during the day

On Monday you had just started. The context window held your request and the files it read.

On Friday you were hours into a session. The window filled, and it compacted. Compaction is not neutral: it clears older tool outputs first, then summarises the conversation. Requests and key snippets survive; detailed instructions from early on may not.

So the careful constraint you gave at ten in the morning was summarised away by two. The prompt did not change. The context around it did.

Claude Code clears older tool outputs first, then summarises the conversation if needed; requests and key code snippets are preserved, and detailed instructions from early in the conversation may be lost.

Claude Code docs, How Claude Code works, checked September 2026

10 The first genuinely senior habit

Put persistent rules in CLAUDE.md, not in the conversation.

That is the documentation's own advice and now you know why. A file read at the start of every session is not part of the conversation being summarised, so it cannot be summarised away.

A rule in the conversation is a rule that expires.

Put persistent rules in CLAUDE.md rather than relying on conversation history.

Claude Code docs, How Claude Code works, checked September 2026

CLAUDE.md is read at the start of every session, which is what makes it survive a compaction that the conversation does not.

First principles, not vendor documentation

Before you blame the tool

1. How long has this session been running?
2. Has it compacted since you gave your key instruction?
3. Is that instruction in CLAUDE.md, or only in the conversation?
4. Which surface are you on, and does it matter for this task?
5. Are you near a five hour window boundary?
6. Have you spent today's pool in the chat app rather than the terminal?
7. Did you install natively, and is it actually up to date?
8. Are you optimising your phrasing when you should be optimising the context?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

☕ Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

Official docs

<https://code.claude.com/docs/en/overview>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.